

Assignment-12.4

Name: K.Bhavya sri

H.no-2303A51863

Batch-13

Task 1: Bubble Sort for Ranking Exam Scores

Scenario: You are working on a college result processing system where a small list of student scores needs to be sorted after every internal assessment.

Task Description

- Implement Bubble Sort in Python to sort a list of student scores.
- Use an AI tool to:
 - o Insert inline comments explaining key operations such as comparisons, swaps, and iteration passes
 - o Identify early-termination conditions when the list becomes sorted
 - o Provide a brief time complexity analysis

Expected Outcome

- A Bubble Sort implementation with:
 - o AI-generated comments explaining the logic
 - o Clear explanation of best, average, and worst-case complexity
 - o Sample input/output showing sorted scores

```
554 def bubble_sort(scores):
555     n = len(scores)
556     for i in range(n):
557         swapped = False
558         for j in range(0, n - i - 1):
559             if scores[j] > scores[j + 1]:
560                 scores[j], scores[j + 1] = scores[j + 1], scores[j]
561                 swapped = True
562         if not swapped:
563             print(f"Early termination on pass {i + 1} (List already sorted)")
564             break
565     return scores
566 # Sample Input
567 scores = [78, 92, 85, 60, 88, 75]
568 print("Original Scores:", scores)
569 sorted_scores = bubble_sort(scores)
570 print("Sorted Scores:", sorted_scores)
```

Output:

```
Original Scores: [78, 92, 85, 60, 88, 75]
Early termination on pass 5 (List already sorted)
Sorted Scores: [60, 75, 78, 85, 88, 92]
```

Justification for the Bubble Sort:

Bubble Sort is simple and easy to implement, making it suitable for small datasets like internal assessment scores.

It repeatedly compares adjacent elements and swaps them if they are in the wrong order.

The algorithm includes an early termination optimization, which improves performance if the list becomes sorted early.

Since the number of students in internal exams is usually small, its $O(n^2)$ complexity is acceptable.

Task 2: Improving Sorting for Nearly Sorted Attendance Records

Scenario: You are maintaining an attendance system where student roll numbers are already almost sorted, with only a few late updates.

Task Description

- Start with a Bubble Sort implementation.
- Ask AI to:
 - o Review the problem and suggest a more suitable sorting algorithm
 - o Generate an Insertion Sort implementation
 - o Explain why Insertion Sort performs better on nearly sorted data
- Compare execution behavior on nearly sorted input

Expected Outcome

- Two sorting implementations:
 - o Bubble Sort
 - o Insertion Sort
- AI-assisted explanation highlighting efficiency differences for partially sorted datasets

Bubble Sort Implementation:

```
576 def bubble_sort(arr):
577     n = len(arr)
578     for i in range(n):
579         swapped = False # Tracks if any swap happens in this pass
580         for j in range(0, n - i - 1):
581             # Compare adjacent roll numbers
582             if arr[j] > arr[j + 1]:
583                 # Swap if out of order
584                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
585                 swapped = True
586             # Early termination if no swaps occur
587             if not swapped:
588                 break
589         return arr
590 # Nearly sorted attendance roll numbers
591 attendance = [101, 102, 103, 105, 104, 106, 107]
592 print("Original Attendance:", attendance)
593 print("Bubble Sort Result:", bubble_sort(arr.copy()))
594
```

Output:

```
Original Attendance: [101, 102, 103, 105, 104, 106, 107]
Bubble Sort Result: [101, 102, 103, 104, 105, 106, 107]
```

Justification for Bubble Sort:

Bubble Sort is simple and easy to implement, making it suitable for small datasets. It repeatedly compares adjacent elements and swaps them if they are out of order. With the early termination optimization, it stops if no swaps occur in a pass. However, it still performs multiple passes, making it less efficient for nearly sorted data.

Insertion Sort Implementation:

```
598
599 def insertion_sort(arr):
600     n = len(arr)
601     for i in range(1, n):
602         key = arr[i] # Current roll number to insert
603         j = i - 1
604         # Shift elements that are greater than key
605         # to one position ahead
606         while j >= 0 and arr[j] > key:
607             arr[j + 1] = arr[j]
608             j -= 1
609         # Place key in correct sorted position
610         arr[j + 1] = key
611     return arr
612 # Nearly sorted attendance roll numbers
613 attendance = [101, 102, 103, 105, 104, 106, 107]
614 print("Original Attendance:", attendance)
615 print("Insertion Sort Result:", insertion_sort(arr.copy()))
```

Output:

```
Original Attendance: [101, 102, 103, 105, 104, 106, 107]
Insertion Sort Result: [101, 102, 103, 104, 105, 106, 107]
```

Justification for Insertion Sort:

Insertion Sort is highly efficient for nearly sorted data because it shifts only misplaced elements. It builds the sorted portion of the list incrementally. When the data is almost sorted, very few shifts are required, leading to faster execution. Its best-case time complexity is $O(n)$, making it more suitable for attendance records with minor updates.

Task 3: Searching Student Records in a Database

Scenario: You are developing a student information portal where users search for student records by roll number.

Task Description

- Implement:
 - o Linear Search for unsorted student data
 - o Binary Search for sorted student data
- Use AI to:
 - o Add docstrings explaining parameters and return values
 - o Explain when Binary Search is applicable
 - o Highlight performance differences between the two searches

Expected Outcome

- Two working search implementations with docstrings
- AI-generated explanation of:
 - o Time complexity
 - o Use cases for Linear vs Binary Search
- A short student observation comparing results on sorted vs unsorted lists

Linear Search (For Unsorted Data):

```
620
621 def linear_search(records, target_roll):
622     for index, roll in enumerate(records):
623         # Compare each element with target roll number
624         if roll == target_roll:
625             return index
626
627     # If not found after checking all elements
628     return -1
629
630 unsorted_records = [105, 101, 109, 103, 107]
631 sorted_records = sorted(unsorted_records)
632 target = 103
633 print("Linear Search (variable) unsorted_records: list[int]")
634 linear_search(unsorted_records, target)
```

Output:

```
Linear Search Result (Unsorted): 3
```

Justification for Linear Search:

Linear Search is suitable for unsorted student records because it does not require any specific order in the data.

It checks each element sequentially until the target roll number is found.

It is simple to implement and works well for small datasets.

However, its time complexity is $O(n)$, making it slower for large databases.

Binary Search (For Sorted Data):

```
639 def binary_search(records, target_roll):
640     left = 0
641     right = len(records) - 1
642     while left <= right:
643         mid = (left + right) // 2
644         # Check if middle element is target
645         if records[mid] == target_roll:
646             return mid
647         # If target is greater, search right half
648         elif records[mid] < target_roll:
649             left = mid + 1
650         # If target is smaller, search left half
651         else:
652             right = mid - 1
653     return -1
654
655 unsorted_records = [105, 101, 109, 103, 107]
656 sorted_records = sorted(unsorted_records)
657 target = 103
658 print("Binary Search Result (Sorted):",
659       binary_search(sorted_records, target))
```

Output:

```
Binary Search Result (Sorted): 1
```

Justification for Binary Search

Binary Search is efficient for sorted student records because it repeatedly divides the search space in half. It significantly reduces the number of comparisons compared to Linear Search. Its time complexity is $O(\log n)$, making it highly efficient for large datasets. However, it can only be applied when the data is sorted.

Task 4: Choosing Between Quick Sort and Merge Sort for Data Processing

Scenario: You are part of a data analytics team that needs to sort large datasets received from different sources (random order, already sorted, and reverse sorted).

Task Description

- Provide AI with partially written recursive functions for:
 - o Quick Sort
 - o Merge Sort
- Ask AI to:
 - o Complete the recursive logic
 - o Add meaningful docstrings
 - o Explain how recursion works in each algorithm
- Test both algorithms on:
 - o Random data
 - o Sorted data
 - o Reverse-sorted data

Expected Outcome

- Fully functional Quick Sort and Merge Sort implementations
- AI-generated comparison covering:
 - o Best, average, and worst-case complexities
 - o Practical scenarios where one algorithm is preferred over the other

Quick Sort Implementation (Recursive):

```
663
664 def quick_sort(arr):
665     if len(arr) <= 1:
666         return arr
667     pivot = arr[-1]
668     # Partitioning step
669     left = [x for x in arr[:-1] if x <= pivot]
670     right = [x for x in arr[:-1] if x > pivot]
671     # Recursive step:
672     return quick_sort(left) + [pivot] + quick_sort(right)
673 random_data = [45, 12, 89, 33, 7, 56]
674 sorted_data = [7, 12, 33, 45, 56, 89]
675 reverse_data = [89, 56, 45, 33, 12, 7]
676
677 print("Quick Sort Results:")
678 print("Random:", quick_sort(random_data))
679 print("Sorted:", quick_sort(sorted_data))
680 print("Reverse:", quick_sort(reverse_data))
```

Output:

```
Quick Sort Results:
Random: [7, 12, 33, 45, 56, 89]
Sorted: [7, 12, 33, 45, 56, 89]
Reverse: [7, 12, 33, 45, 56, 89]
```

Justification for Quick Sort

Quick Sort is preferred for large, randomly ordered datasets because it has an average time complexity of $O(n \log n)$ and performs very fast in practice.

It works by selecting a pivot and partitioning the array into smaller subarrays recursively.

It requires less extra memory since it can be implemented in-place.

However, its worst-case time complexity is $O(n^2)$ when the data is already sorted or reverse sorted (with poor pivot selection).

Merge Sort Implementation (Recursive):

```
668 def merge_sort(arr):
669     if len(arr) <= 1:
670         return arr
671     mid = len(arr) // 2
672     left_half = arr[:mid]
673     right_half = arr[mid:]
674     left_sorted = merge_sort(left_half)
675     right_sorted = merge_sort(right_half)
676     return merge(left_sorted, right_sorted)
677 def merge(left, right):
678     sorted_list = []
679     i = j = 0
680     while i < len(left) and j < len(right):
681         if left[i] < right[j]:
682             sorted_list.append(left[i])
683             i += 1
684         else:
685             sorted_list.append(right[j])
686             j += 1
687     sorted_list.extend(left[i:])
688     sorted_list.extend(right[j:])
689     return sorted_list
```

Output:

```
Merge Sort Results:
Random: [7, 12, 33, 45, 56, 89]
Sorted: [7, 12, 33, 45, 56, 89]
Reverse: [7, 12, 33, 45, 56, 89]
```

Justification for Merge Sort

Merge Sort guarantees a consistent time complexity of $O(n \log n)$ in all cases (best, average and worst). It uses a divide and conquer approach by splitting the array and merging sorted halves recursively. It is stable, meaning it preserves the order of equal elements. However, it requires additional memory $O(n)$ for merging, making it less space-efficient than Quick Sort.

Task 5: Optimizing a Duplicate Detection Algorithm

Scenario: You are building a data validation module that must detect duplicate user IDs in a large dataset before importing it into a system.

Task Description

- Write a naive duplicate detection algorithm using nested loops.
- Use AI to:
 - o Analyze the time complexity and Suggest an optimized approach using sets or dictionaries
 - o Rewrite the algorithm with improved efficiency
- Compare execution behavior conceptually for large input sizes

Expected Outcome

- Two versions of the algorithm: o Brute-force ($O(n^2)$) o Optimized ($O(n)$)
- AI-assisted explanation showing how and why performance Improved

Brute-Force Duplicate Detection ($O(n^2)$):

```
703
704 def find_duplicates_bruteforce(user_ids):
705     duplicates = []
706
707     for i in range(len(user_ids)):
708         for j in range(i + 1, len(user_ids)):
709             # Compare every pair of elements
710             if user_ids[i] == user_ids[j]:
711                 if user_ids[i] not in duplicates:
712                     duplicates.append(user_ids[i])
713     return duplicates
714
715 # Sample dataset with duplicate user IDs
716 user_ids = [101, 205, 309, 101, 450, 205, 678, 900, 309]
717
718 print("Original User IDs:")
719 print(user_ids)
720
721 # Brute-force method
722 duplicates_brute = find_duplicates_bruteforce(user_ids)
723 print("\nDuplicates (Brute Force):")
724 print(duplicates_brute)
```


Output :

```
Merge Sort Results:  
Random: [7, 12, 33, 45, 56, 89]  
Sorted: [7, 12, 33, 45, 56, 89]  
Reverse: [7, 12, 33, 45, 56, 89]
```

Justification for Brute-Force Duplicate Detection:

The brute-force method compares every element with every element using nested loops. It is simple and easy to implement, making it suitable for very small datasets. However, it performs a large number of unnecessary comparisons as the dataset grows. Its time complexity is $O(n^2)$, making it inefficient for large-scale data validation.

Optimized Duplicate Detection Using Set ($O(n)$):

```
708  
709 def find_duplicates_optimized(user_ids):  
710     seen = set()  
711     duplicates = set()  
712  
713     for user_id in user_ids:  
714         # If already seen, it's a duplicate  
715         if user_id in seen:  
716             duplicates.add(user_id)  
717         else:  
718             seen.add(user_id)  
719  
720     return list(duplicates)  
721  
722 # Sample dataset with duplicate user IDs  
723 user_ids = [101, 205, 309, 101, 450, 205, 678, 900, 309]  
724  
725 print("Original User IDs:")  
726 print(user_ids)  
727  
728 # Optimized method  
729 duplicates_optimized = find_duplicates_optimized(user_ids)  
730 print("\nDuplicates (Optimized using Set):")  
731 print(duplicates_optimized)
```

Output:

```
Original User IDs:  
[101, 205, 309, 101, 450, 205, 678, 900, 309]  
  
Duplicates (Optimized using Set):  
[205, 309, 101]
```

Justification for Optimized Duplicate Detection:

The optimized method uses a set to track previously seen user IDs. Set lookup operations take $O(1)$ average time, eliminating repeated comparisons. Each element is processed only once, reducing overall operations. Its time complexity is $O(n)$, making it highly efficient and scalable for large datasets.